A simple AHRS navigation system

1 Introduction

An *Attitude and Heading Reference System* (AHRS) is the system that will estimate the orientation of our drone in three-dimensional space. An AHRS outputs of roll, pitch, and heading (yaw), which describe the attitude of the body relative to a reference frame.

An AHRS typically combines data from gyroscopes, accelerometers, and magnetometers. Through sensor fusion algorithms, these measurements are used to produce a stable and accurate estimate of orientation while mitigating the limitations of individual sensors.

2 Reference Frame Definitions

Two reference frames are used throughout this work: the body-fixed frame (b) and the world-fixed frame (n). The body-fixed frame (b) is attached to the drone and moves with it. Its axes are defined as follows:

- The x-axis points along the forward direction of the drone. Rotation about this axis is referred to as *roll*.
- The y-axis points to the right side of the drone. Rotation about this axis is referred to as *pitch*.
- The z-axis points upward. Rotation about this axis is referred to as *yaw*.
- The coordinate system is right-handed.

The world-fixed frame (n) is an inertial reference frame following the East–North–Up (ENU) convention:

- The x-axis points toward the East.
- The y-axis points toward the North.
- The z-axis points upward.
- The coordinate system is right-handed.

3 The conceptual idea

In this section, we present the conceptual foundation of our AHRS implementation. The system relies on three sensors:

- Gyroscope: measures the angular velocity vector $\boldsymbol{\omega}$ around each axis of the body-fixed frame b.
- Accelerometer: measures the specific force acting on the body as a three-dimensional vector in the body-fixed frame b. When the body is not subject to linear acceleration, the measurement corresponds to the gravity vector.
- Magnetometer: measures the local Earth's magnetic field as a three-dimensional vector in the body-fixed frame b. This measurement provides a reference for determining heading relative to magnetic north.

The gyroscope provides the high-frequency component of the orientation estimate since the measured angular velocity can be integrated to determine the body's orientation over time. However, relying solely on gyroscope measurements is not sufficient, since sensor biases and noise accumulate during integration, causing the estimated orientation to drift over time.

To mitigate this drift, measurements from the accelerometer and magnetometer are incorporated. The accelerometer cannot directly provide the gravity vector during flight, since its measurements are corrupted by the drone's linear accelerations. Nevertheless, under the assumption that these accelerations average to zero over sufficiently long time intervals, the average accelerometer measurement converges to the gravity vector. Similarly, the magnetometer provides a measurement of the Earth's magnetic field, which is assumed to be constant in the earth-fixed frame n.

Consequently, the gravity and magnetic field vectors serve as stable low-frequency references that can be used to correct the long-term drift of the orientation estimate while preserving the gyroscope's fast dynamic response.

4 Description of the Algorithm

4.1 System Initialization

Before the AHRS can reliably track orientation using gyroscope integration, an initial orientation matrix R_0^{nb} must be established. When the drone is powered on and stationary on the ground, the initial acceleration measurement corresponds purely to gravity, and the magnetometer measures the static local Earth's magnetic field. We can exploit these physical vector readings to construct a deterministic, orthogonal coordinate system via the Triad method.

A rotation matrix R^{sd} transforms a vector from a source frame s to a destination frame d. Conceptually, it is structured such that **each column represents a unit basis vector of the source frame s expressed in the coordinates of the destination frame d.** Because our sensor measurements naturally provide the navigation frame's axes (gravity and magnetic north) expressed in body-frame coordinates, we can construct the matrix R_0^{nb} (from navigation n to body b). The columns of R_0^{nb} will simply be the navigation frame's basis vectors (East, North, Up) expressed in body-frame coordinates.

Let a^b_{init} and m^b_{init} be the initial normalized vector measurements of the accelerometer and magnetometer expressed in the body frame:

$$a^b = \frac{a^b_{\text{init}}}{\|a^b_{\text{init}}\|}, \quad m^b = \frac{m^b_{\text{init}}}{\|m^b_{\text{init}}\|}$$

According to the ENU reference frame definitions, the true gravity vector points along the negative z-axis (g-frame), meaning the Up axis (z) points directly opposite to gravity. Crucially, a stationary accelerometer measures *proper acceleration*—the upward reaction force exerted by the ground to oppose gravity. Consequently, the positive body-frame acceleration vector a^b points physically upward, meaning it directly represents the tracking of the navigation frame's Up basis vector ($\mathbf{u}^b$) without requiring negation:

$$\mathbf{u}^b = a^b$$

Next, the raw magnetometer vector m^b cannot be used directly as the North basis vector because it is generally not perfectly orthogonal to the gravity vector due to the magnetic inclination angle (dip angle). To extract a clean East-West lateral axis that is strictly perpendicular to the vertical axis, we compute the cross product of the magnetic vector and the vertical axis. This isolates the navigation frame's East basis vector ($\mathbf{e}^b$) expressed in the body frame:

$$\mathbf{e}^b = \frac{m^b \times \mathbf{u}^b}{\|m^b \times \mathbf{u}^b\|}$$

Finally, to complete the right-handed coordinate system and obtain the true horizontal North basis vector ($\mathbf{n}^b$) expressed in the body frame, we take the cross product of our newly computed Up and East basis vectors:

$$\mathbf{n}^b = \mathbf{u}^b \times \mathbf{e}^b$$

Because $\mathbf{e}^b$, $\mathbf{n}^b$, and $\mathbf{u}^b$ are the mutually orthogonal unit basis vectors of the source frame (n) expressed in the destination frame (b), we pack them directly into the columns of R_0^{nb}:

$$R_0^{nb} = \begin{bmatrix} \mathbf{e}^b & \mathbf{n}^b & \mathbf{u}^b \end{bmatrix}$$

4.2 Computing New Orientation

Mathematically, we describe the orientation of the drone as a matrix R_k^{nb}. This matrix transforms 3D points or vectors described in the world-fixed n coordinate system to the body-fixed b coordinate system. The inverse transformation ($b \to n$) can be done by simply inverting (or transposing, since it is orthogonal) the matrix:

$$R_k^{bn} = (R_k^{nb})^{-1} = (R_k^{nb})^T$$

We can use the elements in this 3×3 matrix to derive the Euler angles (roll, pitch, yaw).

Given the orientation estimate from the previous time step, denoted by R_k^{nb}, the orientation at the next time step can be predicted using the angular velocity measurements provided by the gyroscope. The gyroscope measures the instantaneous angular velocity of the drone expressed in the body-fixed frame b.

Assuming that the angular velocity remains constant during the sampling interval (Δt), the orientation can be propagated forward in time. Because the angular velocity $\boldsymbol{\omega}$ is measured relative to the local body frame, the rotation update must pre-multiply our orientation matrix:

$$R_{k+1}^{nb} = e^{\Omega(\boldsymbol{\omega})\Delta t} R_k^{nb}$$

To make this equation suitable for real-time computation on an embedded system, we can expand the matrix exponential using a Taylor series approximation:

$$e^{\Omega(\boldsymbol{\omega})\Delta t} = I + \Omega(\boldsymbol{\omega})\Delta t + \frac{1}{2!}\left(\Omega(\boldsymbol{\omega})\Delta t\right)^2 + \frac{1}{3!}\left(\Omega(\boldsymbol{\omega})\Delta t\right)^3 + \dots$$

Given that the sampling interval is highly frequent (e.g., $\Delta t = 0.01\,\text{s}$), the higher-order terms ($(\Delta t)^2, (\Delta t)^3, \dots$) become negligibly small. Truncating the series to a first-order linear approximation yields:

$$e^{\Omega(\boldsymbol{\omega})\Delta t} \approx I + \Omega(\boldsymbol{\omega})\Delta t$$

Substituting this back into the propagation step results in a computationally lightweight update equation:

$$R_{k+1}^{nb} \approx \left(I + \Omega(\boldsymbol{\omega})\Delta t\right) R_k^{nb}$$

Where:

- R_{k+1}^{nb} is a 3×3 rotation matrix describing the new uncorrected orientation at time step $k+1$.
- I is the 3×3 identity matrix.
- $\Omega(\boldsymbol{\omega})$ is the skew-symmetric 3×3 matrix encoding the body-frame angular velocity vector $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$:

$$\Omega(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

4.3 Orientation Drift Correction

While gyroscope integration provides smooth and high-frequency orientation estimates, it inevitably accumulates drift due to sensor bias and measurement noise. To compensate for this drift, the integrated orientation is continuously corrected using an absolute orientation estimate obtained from the accelerometer and magnetometer.

The raw accelerometer and magnetometer measurements are normalized to unit length directly at each time step k:

$$\tilde{a}_k^b = \frac{a_k^b}{\|a_k^b\|},$$

$$\tilde{m}_k^b = \frac{m_k^b}{\|m_k^b\|}.$$

These raw normalized measurements are used directly as inputs to the TRIAD algorithm described in the previous section, yielding an absolute orientation estimate

$$R_{\text{triad}}^{nb}.$$

By utilizing the raw measurements directly, the algorithm avoids the phase lag introduced by low-pass filtering, ensuring that the absolute reference alignment responds instantaneously to orientation changes.

Meanwhile, the gyroscope measurements are integrated to propagate the current orientation estimate,

$$R_{\text{gyro}}^{nb}.$$

Although this estimate accurately captures rapid rotational motion, its orientation gradually drifts over time.

To determine the accumulated drift, the rotational difference between the gyroscope estimate and the TRIAD estimate is computed as

$$R_{\text{err}} = R_{\text{triad}}^{nb} \left(R_{\text{gyro}}^{nb}\right)^T.$$

If both orientation estimates coincide, then $R_{\text{err}} = I$, where I denotes the identity matrix. Otherwise, R_{err} represents the rotation required to align the gyroscope estimate with the absolute orientation estimate.

Applying this full correction instantaneously would introduce abrupt changes in the estimated orientation, especially when relying on raw measurements that contain sensor noise and transient disturbances. Instead, only a small fraction of the correction is applied during each update. Let

$$R_{\text{err}} = \exp([\phi]_\times),$$

where $\phi \in \mathbb{R}^3$ is the rotation vector corresponding to the orientation error and $[\cdot]_\times$ denotes the skew-symmetric matrix operator.

To extract the exponent vector $\phi = \theta\mathbf{u}$ from the 3×3 rotation matrix R_{err}, the matrix logarithm is computed via the inverse Rodrigues' formula. The rotation angle θ is derived from the matrix trace:

$$\theta = \arccos\left(\frac{\text{tr}(R_{\text{err}}) - 1}{2}\right),$$

where $\theta \in [0, \pi]$. For nominal conditions where $\theta \neq 0$, the rotation vector is isolated using the off-diagonal elements of R_{err}:

$$\phi = \frac{\theta}{2\sin(\theta)} \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix},$$

where R_{ij} denotes the element at row i and column j. Numerically, when the orientation error approaches zero ($\theta \to 0$), the singularity is bypassed by setting $\phi = [0, 0, 0]^T$ as $\lim_{\theta \to 0} \frac{\theta}{\sin(\theta)} = 1$.

A partial correction is then obtained by scaling this extracted rotation vector:

$$R_{\text{corr}} = \exp(\beta[\phi]_\times),$$

where $0 < \beta \ll 1$ is a correction gain. For example, choosing $\beta = 0.01$ applies approximately one percent of the estimated orientation error during each update, effectively smoothing out high-frequency noise and transient anomalies through the correction gain itself rather than preprocessing the sensor inputs.

Finally, the gyroscope orientation estimate is corrected according to

$$R_{\text{gyro}}^{nb} \leftarrow R_{\text{corr}} R_{\text{gyro}}^{nb}.$$

This gradual correction preserves the smooth, high-frequency response of the gyroscope integration while continuously removing long-term drift. Consequently, the estimated orientation remains stable over extended periods without introducing discontinuities or sacrificing responsiveness.

4.4 Normalization of the Rotation Matrix

The first-order integration method used to propagate the orientation,

$$R_{k+1}^{nb} \approx (I + \Omega(\boldsymbol{\omega})\Delta t)\, R_k^{nb},$$

does not exactly preserve the orthogonality of the rotation matrix. Due to numerical approximation errors, the rows and columns of the matrix gradually lose their unit length and mutual orthogonality. If left uncorrected, the matrix will eventually cease to represent a valid rotation.

To maintain a valid orientation estimate, the rotation matrix must be periodically re-orthogonalized. A computationally efficient method is based on the Gram–Schmidt process.

Let the rows of the estimated rotation matrix be

$$R^{nb} = \begin{bmatrix} r_1^T \\ r_2^T \\ r_3^T \end{bmatrix}.$$

First, the first row is normalized:

$$\hat{r}_1 = \frac{r_1}{\|r_1\|}.$$

Next, the component of the second row along the first row is removed:

$$r_2' = r_2 - (\hat{r}_1 \cdot r_2)\hat{r}_1,$$

and the result is normalized:

$$\hat{r}_2 = \frac{r_2'}{\|r_2'\|}.$$

Finally, the third row is reconstructed using the cross product:

$$\hat{r}_3 = \hat{r}_1 \times \hat{r}_2.$$

The corrected rotation matrix is then

$$R^{nb} = \begin{bmatrix} \hat{r}_1^T \\ \hat{r}_2^T \\ \hat{r}_3^T \end{bmatrix}.$$

This procedure guarantees that the rows remain mutually orthogonal and of unit length, thereby ensuring that the matrix remains a valid rotation matrix satisfying

$$(R^{nb})^T R^{nb} = I.$$

In practice, this normalization step should be performed after each orientation update cycle to prevent the accumulation of numerical errors.

4.5 Extraction of Euler Angles

Once the orientation matrix R_k^{nb} is updated and corrected, the vehicle's attitude can be expressed in terms of traditional Euler angles: roll (ϕ), pitch (θ), and yaw (ψ). Following the standard aerospace convention for an East–North–Up (ENU) navigation frame transforming into a body frame ($n \to b$), the matrix is constructed via a sequential Yaw-Pitch-Roll (Z-Y-X) rotation sequence:

$$R^{nb}(\phi, \theta, \psi) = R_x(\phi) R_y(\theta) R_z(\psi)$$

Evaluating this product yields the algebraic structure of our 3×3 orientation matrix in terms of the individual angles:

$$R^{nb} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} = \begin{bmatrix} \cos\theta\cos\psi & \cos\theta\sin\psi & -\sin\theta \\ \sin\phi\sin\theta\cos\psi - \cos\phi\sin\psi & \sin\phi\sin\theta\sin\psi + \cos\phi\cos\psi & \sin\phi\cos\theta \\ \cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi & \cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi & \cos\phi\cos\theta \end{bmatrix}$$

Under normal flight dynamics where the drone does not experience extreme vertical attitudes, the Euler angles can be isolated directly using the multi-quadrant inverse tangent function (atan2) and the inverse sine function:

$$\begin{aligned} \phi &= \text{atan2}\left(r_{23}, r_{33}\right) \\ \theta &= -\arcsin\left(r_{13}\right) \\ \psi &= \text{atan2}\left(r_{12}, r_{11}\right) \end{aligned}$$

4.5.1 Handling Gimbal Lock

A mathematical singularity known as *Gimbal Lock* occurs if the drone pitches straight up ($\theta = 90°$) or straight down ($\theta = -90°$). In these states, $\cos\theta = 0$, causing the terms r_{11}, r_{12}, r_{23}, and r_{33} to become zero. This collapses roll and yaw onto the same geometric axis, making it impossible to solve for them independently. To prevent numerical instability in the firmware, conditional branching must be applied:

Case 1: Pitching Straight Up ($r_{13} \approx -1$)
The pitch angle is locked to $\theta = \pi/2$. We arbitrarily set the roll angle to zero ($\phi = 0$) to resolve the degree of freedom, allowing yaw to be cleanly computed as:

$$\psi = \text{atan2}\left(r_{21}, r_{22}\right)$$

Case 2: Pitching Straight Down ($r_{13} \approx 1$)
The pitch angle is locked to $\theta = -\pi/2$. Setting the roll angle to zero ($\phi = 0$), yaw is computed as:

$$\psi = -\text{atan2}\left(r_{21}, r_{22}\right)$$